

MetaRoute-Bench: Evaluating Meta-Decision Policies for Agentic Workflows

Natan Vidra
Anote AI
New York, United States
nvidra@anote.ai

Alina Kapanova
Anote AI, Cornell University
New York, United States
ak2765@cornell.edu

Arun Kanhai
Anote AI, Queens College
New York, United States
arun.kanhai21@outlook.com

Abstract

Agentic systems must repeatedly decide whether to answer directly, decompose a task, invoke a tool, execute code, delegate to a specialist, verify an intermediate result, or recover from failure. These meta-decisions affect not only task success but also operating cost and latency, yet they are often embedded inside an orchestration framework and evaluated only through aggregate task accuracy. We present MetaRoute-Bench, an open, inspectable framework for comparing meta-decision policies under a shared execution model. The initial benchmark contains 180 synthetic task profiles spanning data analysis, research, and document processing, eight routing policies, and 30 paired random seeds. Across 43,200 traces, a task-aware compositional policy achieves 79.4% success compared with 76.7% for a strong workload-specific static policy, 67.4% for one-shot task routing, and 52.9% for direct answering. Relative to the static policy, this is a 2.7 percentage-point improvement (paired 95% CI: ± 2.0 points) at 4.7% higher mean cost and 6.4% higher latency. Ablations show the largest losses when route composition is restricted to one operation and when verification is removed. These results are generated by a seeded offline execution model rather than a live deployment; accordingly, our primary contribution is a reproducible evaluation method and an analysis of routing-policy tradeoffs, not evidence of production effectiveness. We release task generation, policies, traces, tests, and analysis artifacts to support live-system validation.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Software and its engineering** → *Software performance*; • **General and reference** → *Evaluation*.

Keywords

agentic systems, routing, orchestration, evaluation, tool use, reproducibility

ACM Reference Format:

Natan Vidra, Alina Kapanova, and Arun Kanhai. 2026. MetaRoute-Bench: Evaluating Meta-Decision Policies for Agentic Workflows. In *8th International Conference on Distributed Artificial Intelligence (DAI '26)*, November 29–December 2, 2026, Hong Kong. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

An agentic application is more than a language model call. It is a distributed decision process involving models, retrieval systems,

code executors, specialist agents, verification stages, retry logic, and human escalation. Before any component can help, the system must decide which component to invoke and when. A poor meta-decision can waste time on unnecessary decomposition, call an irrelevant tool, execute unsafe or unhelpful code, or stop before a result has been checked.

Existing research has established the value of interleaving reasoning and action [17], learning API use [11, 12], routing among models [1, 9], and optimizing function-call plans [5]. Agent benchmarks reveal persistent failures in long-horizon reasoning and decision making [8], while domain benchmarks increasingly emphasize executable outcomes [4, 16]. However, system builders still lack a small, inspectable protocol for isolating the policy that chooses among reasoning modes and measuring its success, cost, and latency tradeoffs.

This paper introduces MetaRoute-Bench, an offline benchmark and trace format for that meta-decision layer. We ask: *under controlled execution assumptions, does task-aware adaptive routing improve operational outcomes over fixed and one-shot policies?* Our contributions are:

- a typed framework separating task profiles, routing policies, execution, traces, and evaluation;
- a balanced suite of 180 synthetic profiles across three operational workload families;
- a paired comparison of eight policies over 30 seeds and 43,200 traces; and
- ablations and workload-level analyses identifying when composition, verification, decomposition, and recovery affect outcomes.

The implementation, benchmark profiles, experiment scripts, raw traces, and reproducibility materials are available at <https://github.com/anote-ai/research-coageneration>.

The benchmark is intentionally transparent, but currently simulated. This boundary is central: the reported values validate the framework’s reproducibility and diagnostic behavior, while live models, tools, and organizational workloads are required to establish external validity.

2 Related Work

Adaptive routing and efficient inference. Prior routers establish that computational strategy should vary with the request rather than remain fixed. Adaptive-RAG classifies question complexity and selects among no retrieval, single-step retrieval, and iterative retrieval, improving the accuracy–efficiency balance [3]. CP-Router similarly uses uncertainty to choose between a standard language model and a longer-reasoning model [13]. FrugalGPT and RouteLLM route requests among models to balance quality and

cost [1, 9], while ZeroRouter explicitly optimizes accuracy, cost, and latency and supports onboarding unseen models [15]. These systems motivate our task-difficulty thresholds and separate reporting of success, cost, and latency. MetaRoute-Bench differs by routing among workflow operations rather than only retrieval modes or model endpoints; its current threshold policy is transparent and hand specified rather than learned or uncertainty calibrated.

Planning, decomposition, and verification. ReAct interleaves reasoning and environment actions so plans can evolve from observations [17]. ACPBench uses formal planning domains to synthesize scalable tasks with provably correct answers and shows that current models retain uneven planning abilities [6]. SPIRAL assigns proposing, simulation, and critique to specialized agents inside grounded reflective search [18]. Together, these works support treating decomposition and verification as separable operations and motivate our ablations of each. Our implementation is intentionally less ambitious: it composes an annotated route once and permits an executor-level retry, but does not perform search or observation-conditioned replanning.

Tool selection and execution. Toolformer learns whether, when, and how to invoke APIs [12], and ToolLLM scales tool learning to thousands of real APIs [11]. API-Bank separates planning, API retrieval, and API calling in a runnable benchmark [7]; RESTful-Llama demonstrates an industry-oriented path from natural-language requests and API documentation to REST calls [14]. AnyTool adds hierarchical retrieval and reflection [2], while LLMCompiler optimizes function-call plans for latency and cost [5]. This literature justifies representing tool use, code execution, delegation, and verification as explicit trace events with independent costs and failures. Unlike API-Bank or RESTful-Llama, the present study simulates those events and therefore cannot establish live tool-call robustness.

Agent evaluation and trajectory diagnosis. AgentBench evaluates agents across interactive environments and identifies long-horizon reasoning and decision-making failures [8]. SWE-bench grounds coding-agent evaluation in repository issues [4], and τ -bench evaluates conversational tool agents under domain policies [16]. AgentDiagnose argues that final success alone obscures decomposition, observation reading, verification, and backtracking behavior, and instead analyzes full trajectories [10]. These findings directly motivate our typed traces, failure taxonomy, action-frequency analysis, and workload slices. MetaRoute-Bench contributes a controlled policy-comparison layer; it complements rather than replaces benchmarks with executable tasks and real environments.

3 Framework

3.1 Design Requirements

MetaRoute-Bench is designed around four requirements derived from operational agent evaluation. *Policy isolation* requires routing logic to be interchangeable without changing tasks or the executor. *Paired evaluation* requires every policy to encounter the same profiles and seed schedule. *Trace completeness*, consistent with trajectory-level diagnosis [10], requires the artifact to retain actions, failures, retries, cost, and latency rather than only final correctness. Finally, *assumption visibility* requires simulator parameters to remain inspectable and configurable. These requirements

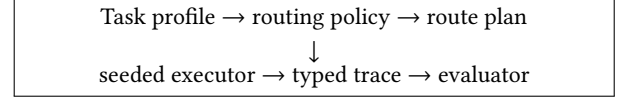


Figure 1: MetaRoute-Bench separates task generation, route selection, execution, and evaluation. Live adapters can replace the seeded executor while preserving trace and metric interfaces.

make the artifact useful for controlled diagnostics today and for later shadow evaluation against live systems.

The unit of evaluation is a route, not an isolated model response. This distinction matters because two systems can use the same base model yet produce different operational behavior through decomposition, tool access, verification, and retry policies. Conversely, a more capable model can be operationally inferior if its controller invokes costly components unnecessarily. MetaRoute-Bench therefore treats the controller as an independently testable system component.

3.2 Meta-Decision Process

Each task x has a workload, difficulty, ambiguity, operation-need annotations, and cost and latency budgets. The present policies map task metadata to a complete route r :

$$r = \pi(x), \quad r \subseteq \{D, T, C, G, V, A\}, \quad (1)$$

where D denotes decomposition, T tool use, C code execution, G delegation, V verification, and A final answering. Every route must end in A ; operations are unique in the current implementation. A trace records actions, success, expected success, cost, latency, retries, budget compliance, confidence, and failure mode. The framework can be extended to policies over execution history, but the evaluated policy adapts only through one executor-level retry after a failed operation; it does not replan from arbitrary intermediate outputs.

We report success, cost, and latency separately. For ranking in the command-line summary only, we define a secondary utility

$$U = S - 0.025K - 0.0015L, \quad (2)$$

where S is success rate, K mean normalized cost, and L mean latency in seconds. Conclusions do not depend solely on this weighting.

3.3 System Architecture

The implementation separates five interfaces: (1) a deterministic workload generator; (2) routing policies; (3) a seeded offline executor; (4) typed trace export; and (5) aggregate and paired evaluation. This separation permits replacement of the simulator with live model and tool adapters without changing policy or analysis interfaces.

The executor makes its assumptions explicit. Each operation has a normalized cost, latency, benefit scaled by task need, and—for external tools, code, and delegation—a failure probability. Difficulty and ambiguity reduce base success. Unnecessary operations impose overhead. Adaptive recovery makes one retry after an execution

failure. All randomness is keyed by seed, task identifier, and policy name.

3.4 Policies

We evaluate four fixed policies (direct, always decompose, always tool, and always code), a random route policy, a workload-specific static rule table, a one-shot router selecting the highest annotated operation need, and the proposed task-aware compositional policy, labeled *adaptive* in the artifacts. It selects up to three operations above a difficulty-dependent threshold, orders decomposition first and verification last, and enables one recovery attempt. Both one-shot and adaptive policies receive the same task-level need annotations; their comparison therefore isolates route composition and recovery rather than feature prediction.

The adaptive policy uses a threshold of .66 for difficulty levels one and two and .56 for levels three and four. This coarse complexity conditioning follows the same design principle as Adaptive-RAG [3], although our thresholds are fixed rather than classifier learned. If no operation clears the threshold, the policy selects the highest-scoring operation. A maximum of three support operations prevents unbounded orchestration overhead. Decomposition is moved to the beginning of a route because it affects downstream work allocation, while verification is moved to the end because it evaluates the assembled result. The current policy is deliberately transparent: every decision can be reconstructed from exported task annotations and constants. This favors auditability over model flexibility and provides a reproducible baseline for future learned or uncertainty-aware routers [13, 15].

The static workload baseline is intentionally strong. Data-analysis profiles execute code and verify, research profiles decompose and use a tool, and document-processing profiles use a tool and verify. It represents the kind of rule table an engineering team might deploy before investing in a learned or task-aware controller. The one-shot policy receives richer task annotations but can choose only one support operation. Comparing these policies distinguishes three questions: whether orchestration helps at all, whether workload rules are sufficient, and whether composing multiple task-specific operations adds value.

4 Evaluation

4.1 Workloads and Protocol

The suite contains 60 task profiles for each of data analysis, research, and document processing. Profiles cover four difficulty levels and use seeded variation around workload-specific needs. Data analysis emphasizes code and verification, research emphasizes decomposition and tools, and document processing emphasizes tools and verification. These profiles represent workflow characteristics rather than natural-language task instances.

Table 1 summarizes the resulting suite. The profiles are balanced by workload and difficulty, but intentionally heterogeneous within each workload: every need receives uniform jitter of up to .25 before clipping. This prevents the workload label from fully determining the best route and creates cases where a workload-level rule is unnecessarily expensive or omits a useful operation. Scalable synthetic evaluation has precedent in planning benchmarks such as ACPBench [6]; however, our annotations are not backed by formal

Table 1: Mean annotations for the 180 task profiles. Dcmp. denotes decomposition and Verif. denotes verification.

Workload	Amb.	Dcmp.	Tool	Code	Deleg.	Verif.
Data analysis	.40	.50	.35	.76	.32	.64
Research	.39	.67	.78	.19	.56	.67
Document proc.	.38	.39	.65	.46	.25	.74

Table 2: Main benchmark results. CI is the 95% interval half-width for success over 30 seeds.

Policy	Success	CI	Cost	Lat. (s)
Adaptive	.794	.012	2.91	20.45
Static workload	.767	.013	2.78	19.23
One-shot	.674	.012	1.83	12.46
Always tool	.637	.011	1.96	13.44
Random	.635	.012	2.54	17.86
Always decompose	.610	.013	1.34	8.40
Always code	.608	.011	2.24	15.68
Direct	.529	.012	.73	4.48

semantics or provably correct plans. They should be viewed as controlled routing signals rather than a realistic task-understanding stage.

We execute every policy on every profile for 30 paired seeds. This produces 5,400 traces per policy and 43,200 main-experiment traces. We report mean success and a 95% confidence interval computed over seed-level success rates, plus mean normalized cost, latency, and cost per success. Paired differences use the same seed-level aggregation. The complete trace file is exported as CSV.

4.2 Main Results

Table 2 shows a clear operational tradeoff. Adaptive routing has the highest success rate, but direct answering is least expensive and fastest. Against the strongest baseline, static workload routing, adaptive routing improves success by 2.74 points (paired 95% CI ± 1.96), while increasing cost by 0.13 units and latency by 1.22 seconds. Compared with one-shot routing, adaptive routing improves success by 12.02 points (± 1.87), with 1.09 additional cost units and 8.00 additional seconds.

Figure 2 makes the absence of a single universally best policy explicit. Direct answering and always-decompose are attractive when latency or cost dominates, while adaptive and static routing occupy the high-success region. The adaptive policy’s cost per successful task is 3.67 units, compared with 3.63 for static routing and 2.71 for one-shot routing. Thus, its success advantage over static routing does not translate into lower cost per success under the current weights. A deployment should select a policy from this frontier using service-level objectives rather than ranking by success alone.

Workload-level adaptive success is .802 for data analysis, .835 for research, and .746 for document processing. The static policy is particularly competitive for data analysis (.791), but trails more on research (.786 versus .835). This suggests that task-level composition

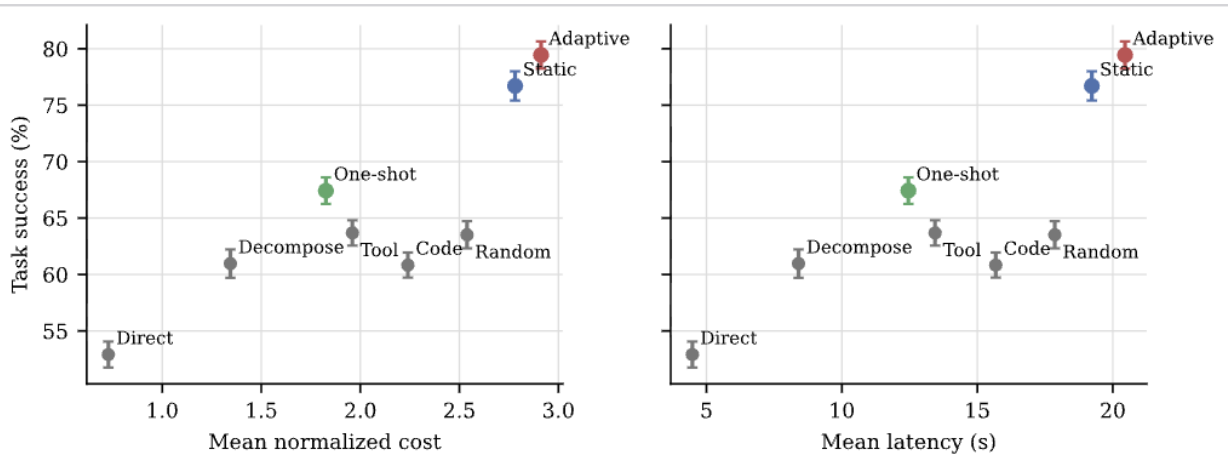


Figure 2: Success versus mean normalized cost (left) and mean latency (right). Error bars show 95% confidence intervals over seed-level success. No policy dominates all three dimensions: direct answering is cheapest and fastest, while adaptive routing has the highest success.

Table 3: Adaptive-policy ablations over 5,400 traces each.

Policy	Success	Cost	Lat. (s)
Adaptive	.794	2.91	20.45
No recovery	.783	2.86	19.98
No decomposition	.769	2.88	20.44
No verification	.747	2.69	18.96
Single operation	.681	1.85	12.68

has the most value where decomposition and information access interact.

4.3 Ablations

Restricting routes to one operation produces the largest reduction: 11.30 points (± 1.85). Removing verification reduces success by 4.70 points (± 1.68), and removing decomposition reduces it by 2.57 points (± 1.68). The 1.13-point recovery difference has an interval of ± 1.70 and therefore does not support a confident recovery benefit in this experiment. This is a useful negative finding: recovery is operationally plausible, but the current failure frequency and sample design do not establish its effect.

4.4 Routing and Failure Analysis

The adaptive policy verifies 68.3% of routes, uses a tool in 50.6%, executes code in 33.3%, decomposes 28.3%, and delegates 13.3%. In contrast, the static policy applies exactly two support operations to every task. The adaptive policy therefore does not improve by simply constructing longer routes; it reallocates operations according to profile-level signals and uses three operations only where multiple needs clear the threshold.

Failure traces separate unrecovered execution failures from ordinary task failures. Adaptive routing records 109 unrecovered tool, code, or delegation failures among 5,400 traces (2.0%), compared with 408 tool or code failures for static routing (7.6%). Ordinary task

failures are similar: 1,060 for adaptive and 1,050 for static. The lower unrecovered-execution count is consistent with retry behavior, but the no-recovery ablation’s confidence interval includes zero. We therefore treat this pattern as diagnostic evidence about the trace model, not proof that recovery improves overall success.

Document processing is the weakest adaptive workload at .746 success. Its high verification and tool needs make routes vulnerable to external-operation failure while offering less benefit from decomposition. Research shows the largest advantage over static routing: 4.89 points. These differences illustrate why aggregate scores should be accompanied by workload slices before an orchestration policy is deployed broadly.

5 Industry Application and Lessons

First, a strong static policy deserves inclusion in agent evaluations. It captures much of the benefit of adaptive routing and is simpler to inspect and operate. Second, success improvements must be reported alongside cost and latency: the adaptive policy is neither cheapest nor fastest. Third, route composition matters more than any single fixed action in this model. Fourth, structured traces make policy behavior auditable; aggregate accuracy alone cannot reveal unnecessary calls, retry behavior, or workload-specific regressions.

5.1 Integration Pattern

For deployment, the framework should sit above existing model and tool adapters. A task-intake service would provide observable metadata and available capabilities; the router would return a structured route plan; an executor would enforce permissions and budgets; and the trace service would record decisions, outcomes, and stop reasons. This architecture does not require storage of private chain-of-thought. Short structured rationales, confidence values, tool inputs and outputs, timing, and failure codes are sufficient for policy analysis.

The separation between router and executor is operationally important. The router may recommend code execution or delegation,

but the executor remains responsible for sandboxing, access control, timeouts, data residency, and allowlists. Hard controls should not depend on the router’s language-model judgment. Likewise, verification should use an independent check where possible rather than asking the same component to approve its own output.

5.2 Deployment Protocol

A production evaluation should proceed in four stages. First, replay historical, consented traces offline and compare proposed routes with the incumbent policy. Second, run in shadow mode: generate route decisions without executing them, then estimate disagreement, expected cost, and policy coverage. Third, execute only low-risk tasks under hard cost and latency limits, retaining a static fallback and human escalation. Finally, use a randomized or stepped-wedge comparison where organizational constraints permit, measuring end-to-end task success rather than proxy judgments alone.

Operational acceptance criteria should be specified before testing. Examples include a lower bound on task success, a maximum p95 latency, a cost-per-success ceiling, and a maximum unrecovered tool-failure rate. Results should also be sliced by workload, difficulty, data sensitivity, and route type. A global average can hide a policy that is beneficial for research tasks but harmful for document workflows.

5.3 Practical Lessons

The study yields four immediate lessons. First, strong static policies are credible production baselines, not strawmen. Second, route composition can improve success, but each additional operation consumes budget and expands the failure surface. Third, verification appears valuable in the simulator and should be isolated in live ablations rather than assumed beneficial. Fourth, complete traces turn routing into an observable engineering problem: teams can inspect unnecessary calls, failure recovery, policy disagreement, and workload regressions instead of debugging from final responses alone.

6 Limitations, Ethics, and Next Steps

The primary limitation is external validity. Outcomes are sampled from a hand-specified execution model whose operation benefits depend on the same need dimensions supplied to task-aware policies. The adaptive policy should therefore be interpreted as an annotated upper-bound controller, not a learned router. Cost units and latency values are normalized assumptions, not invoices or wall-clock measurements. Task profiles do not contain natural-language inputs, live APIs, concurrent agents, security constraints, or human judgments.

Construct validity is also limited. “Success” is a Bernoulli outcome generated from the simulator rather than an independently graded artifact, and the confidence intervals quantify sampling variation under the fixed model, not uncertainty about the model assumptions themselves. The utility weights are illustrative and can change policy rankings. Internal validity is stronger because policies share task profiles, seeds, budgets, and executor logic, but policy-specific random streams mean traces are paired at the seed

aggregate rather than by identical random draws for every operation.

The generated workload profiles encode our expectations about which operations help each task family. This makes the benchmark appropriate for checking whether evaluation machinery behaves coherently, but it also creates a form of evaluator-policy alignment. A live study must derive routing signals from task text or operational metadata without revealing ground-truth operation utilities. It should also test distribution shift, missing tools, correlated failures, concurrent execution, and adversarial or malformed inputs.

These limitations prevent claims of deployment readiness. The next evaluation must replace task annotations with predictions derived from task text, replay real anonymized traces, and then run controlled live tools under fixed budgets. A learned router, model-routing baselines, calibration analysis, and human escalation should also be added. We publish all simulator constants and raw traces so these assumptions can be challenged rather than hidden.

Agent routing can create privacy, security, and accountability risks when tasks are delegated to external services or code is executed. Practical deployments need data-minimization rules, sandboxed execution, allowlisted tools, access control, audit logs, and human review for high-impact actions. The current benchmark performs no real external action and contains no personal or proprietary data.

7 Conclusion

MetaRoute-Bench isolates the orchestration policy governing how an agentic system decomposes, routes, executes, verifies, and recovers. In a reproducible offline study, task-aware adaptive composition improves success over fixed and one-shot policies while incurring measurable cost and latency. The framework and traces provide a foundation for the more consequential next step: validation on live, production-representative workflows.

Acknowledgments

This work was conducted with organizational support from Anote AI. We thank the Anote AI research and engineering team for research infrastructure, technical feedback, and support in developing the reproducible evaluation artifact. No human participants or proprietary data were used in the reported offline benchmark.

References

- [1] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=cSimKw5p6R>
- [2] Yu Du, Fangyun Wei, and Hongyang Zhang. 2024. AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Calls. In *Proceedings of the 41st International Conference on Machine Learning*. <https://openreview.net/forum?id=qFLLbkTQWw>
- [3] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 7036–7050. doi:10.18653/v1/2024.naacl-long.389
- [4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=VTF8yNQm66>

- [5] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 24370–24391. <https://proceedings.mlr.press/v235/kim24y.html>
- [6] Harsha Kokel, Michael Katz, Kavitha Srinivas, and Shirin Sohrabi. 2025. ACP-Bench: Reasoning About Action, Change, and Planning. *Proceedings of the AAAI Conference on Artificial Intelligence* 39, 25 (2025), 26559–26568. doi:10.1609/aaai.v39i25.34857
- [7] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3102–3116. doi:10.18653/v1/2023.emnlp-main.187
- [8] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=zAdUB0aCTQ>
- [9] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs with Preference Data. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=8sSqNntaMr>
- [10] Tianyue Ou, Wanyao Guo, Apurva Gandhi, Graham Neubig, and Xiang Yue. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 207–215. doi:10.18653/v1/2025.emnlp-demos.15
- [11] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=dHng2O0Jjr>
- [12] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*, Vol. 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html
- [13] Jiayuan Su, Fulin Lin, Zhaopeng Feng, Han Zheng, Teng Wang, Zhenyu Xiao, Xinlong Zhao, Zuozhu Liu, Lu Cheng, and Hongwei Wang. 2026. CP-Router: An Uncertainty-Aware Router Between LLM and LRM. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 39 (2026), 33065–33073. doi:10.1609/aaai.v40i39.40589
- [14] Han Xu, Ruining Zhao, Jindong Wang, and Haipeng Chen. 2024. RESTful-Llama: Connecting User Queries to RESTful APIs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, 1433–1443. doi:10.18653/v1/2024.emnlp-industry.105
- [15] Cheng Yan, Wuyang Zhang, Zhiyuan Ning, Fan Xu, Ziyang Tao, Lu Zhang, Bing Yin, and Yanyong Zhang. 2026. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 43 (2026), 36483–36490. doi:10.1609/aaai.v40i43.40970
- [16] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045* (2024). <https://arxiv.org/abs/2406.12045>
- [17] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X
- [18] Yifan Zhang, Giridhar Ganapavarapu, Srideepika Jayaraman, Bhavna Agrawal, Dhaval Patel, and Achille Fokoue. 2026. SPIRAL: Symbolic LLM Planning via Grounded and Reflective Search. *Proceedings of the AAAI Conference on Artificial Intelligence* 40, 43 (2026), 36527–36535. doi:10.1609/aaai.v40i43.40975

A Execution Model

The offline executor uses explicit per-operation assumptions, listed in Table 4. Difficulty multiplies cost and latency by $1 + 0.08(d - 1)$. Base success is $0.69 - 0.075(d - 1) - 0.13a$, where d is difficulty and a is ambiguity. Each selected operation adds a benefit proportional to its annotated task need and a small penalty when unnecessary. Routes longer than two support operations incur coordination overhead. External operations can fail; the adaptive condition permits one retry with additional cost and latency. Probabilities are clipped to $[0.02, 0.98]$. The implementation in `src/metarouter/simulator.py` is authoritative.

Table 4: Offline executor parameters. Failure probability is zero where omitted.

Operation	Cost	Latency (s)	Failure
Decompose	.55	3.5	–
Use tool	1.10	8.0	.08
Execute code	1.35	10.0	.07
Delegate	1.65	13.0	.10
Verify	.70	5.0	–
Answer	.65	4.0	–

B Reproducibility

The artifact requires Python 3.10 or later. The following commands regenerate the main and ablation results and verify that the headline manuscript values match the exported CSV files.

```
python -m venv .venv
source .venv/bin/activate
pip install -e ".[dev]"
metarouter-benchmark --seeds 30 \
    --output results/metarouter
python scripts/run_metarouter_ablations.py
python scripts/check_paper_results.py
pytest -q
```

The artifact exports task profiles, raw traces, policy summaries, paired comparisons, workload-level success, action counts, and failure distributions. The random stream is keyed by seed, task identifier, and policy name.